

1. Modernize Frontend Build & Offline PWA

Target Goal: Transform the frontend into an installable Progressive Web App (PWA) that loads in under 200 milliseconds, functions 100% offline in rural farm plots, and requires zero app store approvals.

WHY THIS MATTERS FOR SMALL FARMERS

In rural areas, farmers frequently experience 2G/3G connections or complete signal blackouts in the middle of their field. A traditional website that downloads a heavy bundle every visit will fail. By using Vite + Service Workers, the entire application shell, diagnosis library, and offline crop doctor remain fully functional without internet.

Step 1: Scaffolding with Vite + React + TypeScript

Open your terminal and run the standard Vite initializer. This sets up lightning-fast Hot Module Replacement (HMR) and production bundling:

```
# 1. Create the project
npm create vite@latest cropcare-pwa -- --template react-ts

# 2. Enter directory and install core dependencies
cd cropcare-pwa
npm install lucide-react recharts idb-keyval

# 3. Install Tailwind CSS and PWA plugin
npm install -D tailwindcss postcss autoprefixer vite-plugin-pwa
npx tailwindcss init -p
```

Step 2: Configure vite.config.ts with PWA Plugin

Replace your vite.config.ts with the following configuration. It sets up automatic Service Worker caching and an installable Web Manifest:

```

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'apple-touch-icon.png', 'leaf-icon.svg'],
      manifest: {
        name: 'CropCare AI - Farmer Health Assistant',
        short_name: 'CropCare AI',
        description: 'AI Crop Disease Detection & Care for Small Farmers',
        theme_color: '#123524',
        background_color: '#F5F7F3',
        display: 'standalone',
        orientation: 'portrait',
        start_url: '/',
        icons: [
          {
            src: 'pwa-192x192.png',
            sizes: '192x192',
            type: 'image/png'
          },
          {
            src: 'pwa-512x512.png',
            sizes: '512x512',
            type: 'image/png',
            purpose: 'any maskable'
          }
        ]
      }
    ]
  },
  workbox: {
    // Cache all local static assets
    globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],
    runtimeCaching: [
      {
        // Cache Google Fonts for offline use
        urlPattern: /^https:\/\/fonts\.googleapis\.com\/.*\/i/,
        handler: 'CacheFirst',
        options: {
          cacheName: 'google-fonts-cache',
          expiration: { maxEntries: 10, maxAgeSeconds: 60 * 60 * 24 * 365 }
        }
      },
      {
        // Cache weather API with network fallback
        urlPattern: /^https:\/\/api\.open-meteo\.com\/.*\/i/,
        handler: 'NetworkFirst',
        options: {
          cacheName: 'weather-cache',
          networkTimeoutSeconds: 3,
          expiration: { maxEntries: 20, maxAgeSeconds: 60 * 60 * 4 } // 4 hours
        }
      }
    ]
  }
});

```

Step 3: Storing Scans Offline via IndexedDB (idb-keyval)

Do not use `localStorage` for leaf photos because it has a 5MB limit. Use IndexedDB so the farmer can save hundreds of offline high-resolution leaf scans directly on device:

```
// src/services/offlineStorage.ts
import { get, set, entries } from 'idb-keyval';

export interface OfflineScanRecord {
  id: string;
  timestamp: number;
  crop: string;
  imageBlob: Blob;
  predictedDisease?: string;
  severity?: number;
  synced: boolean;
}

export async function saveOfflineScan(scan: OfflineScanRecord) {
  await set(`scan_${scan.id}`, scan);
  console.log('Saved scan offline in IndexedDB:', scan.id);
}

export async function getAllOfflineScans(): Promise<OfflineScanRecord[]> {
  const all = await entries();
  return all
    .filter(([k]) => typeof k === 'string' && k.startsWith('scan_'))
    .map([, v]) => v as OfflineScanRecord;
}
```

Step 4: Build & Test Verification

1. Run `npm run build`. Notice the output creates a `dist/sw.js` Service Worker file.
2. Run `npm run preview` to test the production build locally.
3. Open Chrome DevTools $\rightarrow$ **Application** Tab $\rightarrow$ **Service Workers**. Verify status is *Active and running*.
4. Check the **Offline** checkbox in DevTools Network tab and reload the page. The app must load immediately!